Language models

Last modified: 2026-10-30

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission; this page covers the part of his module on recurrent networks that deals with language modeling. The exercises are new.

1 Welcome

1.1 Plan for this lecture

The recurrent networks chapter read whole sequences and classified them, and ended by turning symbols into vectors with an embedding layer. This lecture predicts text one character at a time:

1. the probability of a string, built from the probability of each next character;
2. a model that counts pairs of characters;
3. a number that scores how well a model predicts held-out text;
4. a recurrent network that predicts each next character;
5. drawing new text from that network.

1.2 The text

```
import math
import re

import torch
from torch import nn

text = open("data/shakespeare.txt").read()
chars = sorted(set(text))
n_chars = len(chars)
index = {ch: i for i, ch in enumerate(chars)}
ids = torch.tensor([index[ch] for ch in text])
n_train = 90_000
train_ids, val_ids = ids[:n_train], ids[n_train:]


def decode(t):
    return "".join(chars[i] for i in t.tolist())


print(n_chars, len(text))
```

```
61 100000
```

```
print(text[:120])
```

```
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved ra
```

The exercises use a local copy, `data/shakespeare.txt`[1], of the first 100,000 characters of the tiny Shakespeare text[2]. Karpathy [1] collected that text from Shakespeare's plays. `tools/get-shakespeare.py` rebuilds the copy. The script keeps only the first 100,000 characters, the opening of *Coriolanus*, and changes nothing else. The copy has 61 distinct characters, counting capitals, punctuation, the space and the line break separately. `ids` numbers each character by its place in the sorted list `chars`. The first 90,000 characters are for training, and the remaining 10,000 for validation.

2 Counting

2.1 The probability of a string

Exercise 2.1 (The probability of a word)**.** Let X_1, X_2, X_3 be the next three characters of a text. Use the chain rule of probability[a] to write $\Pr(X_1 = \texttt{t}, X_2 = \texttt{h}, X_3 = \texttt{e})$ as a product of three probabilities, each about one character.

[a] https://morrison-lab.github.io/lds/chapters/probability.html#thm-chain-rule

Solution

Solution 2.1. The chain rule of probability[a] conditions each character on the ones before it:

$$\begin{aligned}&\Pr(X_1 = \texttt{t}, X_2 = \texttt{h}, X_3 = \texttt{e})\\ &\quad = \Pr(X_1 = \texttt{t})\Pr(X_2 = \texttt{h} \mid X_1 = \texttt{t})\Pr(X_3 = \texttt{e} \mid X_1 = \texttt{t}, X_2 = \texttt{h}).\end{aligned}$$

Each factor is the probability of one character given the characters before it. A model that gives these conditional probabilities for every string of earlier characters gives the probability of every string.

[a] https://morrison-lab.github.io/lds/chapters/probability.html#thm-chain-rule

Definition 2.1 (Language model)**.** A **language model** gives, for each position t in a sequence of symbols $x_1, \ldots, x_n$, a probability distribution $\Pr(X_t = x \mid x_1, \ldots, x_{t-1})$ over the symbol x at that position, given the symbols before it [2, Sec. 12.4]. By the chain rule of probability[a], the product of these probabilities over $t = 1, \ldots, n$ is the probability of the whole sequence. The symbols, often called tokens, can be characters or words; these notes use characters.

[a] https://morrison-lab.github.io/lds/chapters/probability.html#thm-chain-rule

Example 2.1 (A uniform language model)**.**

```
uniform_the = (1 / n_chars) ** 3
print(uniform_the)
```

[1] https://morrison-lab.github.io/dds/chapters/data/shakespeare.txt
[2] https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

```
4.40565509888493e-06
```
The simplest language model for this text gives each of the 61 characters probability 1/61 at every position, whatever came before. It gives `"the"` probability $(1/61)^3$, or 1 in 226,981, the same as any other string of three characters, such as `"qzx"`.

2.2 Counting pairs

Exercise 2.2 (What follows a q)**.** In the training text, what share of the `q`s are followed by a `u`?

Solution

Solution 2.2.

```
after_q = train_ids[1:][train_ids[:-1] == index["q"]]
n_q = len(after_q)
share_u = (after_q == index["u"]).float().mean().item()
print(n_q, decode(after_q))
```

```
32 uuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuu
```
100% of the 32 `q`s in the training text are followed by a `u`. So the share of `q`s followed by `u` estimates $\Pr(X_t = \texttt{u} \mid X_{t-1} = \texttt{q})$ as 1.00.

Definition 2.2 (Bigram model)**.** A **bigram model** is a language model in which the distribution of each symbol depends only on the symbol before it [2, Sec. 12.4.1]. Counting in a training text estimates it as

$$\widehat{\Pr}[](X_t = b \mid X_{t-1} = a) = \frac{N(a,b)}{N(a)},$$

where $N(a,b)$ counts how often a is followed by b, and $N(a)$ counts how often a is followed by any symbol [3, Sec. 3.1.2]. An n-gram model conditions on the $n-1$ symbols before each one in the same way.

Example 2.2 (A bigram model of the training text)**.**

```
pair_counts = torch.zeros(n_chars, n_chars)
_ = pair_counts.index_put_(
    (train_ids[:-1], train_ids[1:]), torch.ones(n_train - 1), accumulate=True
)
row_totals = pair_counts.sum(dim=1, keepdim=True)
counted = pair_counts / row_totals
after_t = counted[index["t"]]
top_after_t = chars[after_t.argmax().item()]
p_top_after_t = after_t.max().item()
p_u_after_q = counted[index["q"], index["u"]].item()
print(repr(top_after_t), p_top_after_t, p_u_after_q)
```

```
'h' 0.33165013790130615 1.0
```
Row a of `pair_counts` holds $N(a,b)$ for every b. So row a of `counted` is the bigram model's distribution of the character after a. Its most likely character after `t` is h, with probability 0.33. It gives a `u` after a `q` probability 1.00, the share found in Solution 2.2.

2.3 Pairs never seen

Exercise 2.3 (Pairs the training text never shows)**.** At how many positions in the validation text is a character followed by one that never follows it in the training text?

Solution

Solution 2.3.

```
seen = pair_counts > 0
n_unseen = (~seen[val_ids[:-1], val_ids[1:]]).sum().item()
print(n_unseen)
```

```
12
```

At 12 of the 9,999 positions after the first, the pair of characters never occurs in the training text. The counting estimate of Example 2.2 gives each of those 12 pairs probability 0, and so the whole validation text probability 0.

Definition 2.3 (Add-one smoothing (Laplace smoothing))**. Add-one smoothing** (also called **Laplace smoothing**) adds 1 to every count before estimating a bigram model. With K possible symbols, the estimate becomes

$$\widehat{\Pr}[](X_t = b \mid X_{t-1} = a) = \frac{N(a,b)+1}{N(a)+K},$$

so every pair gets a probability above 0, including pairs the training text never shows [3, Sec. 3.6.1].

Example 2.3 (The smoothed bigram model)**.**

```
bigram = (pair_counts + 1) / (row_totals + n_chars)
p_u_after_q_smoothed = bigram[index["q"], index["u"]].item()
p_unseen_min = bigram[~seen].min().item()
print(p_u_after_q_smoothed, p_unseen_min)
```

```
0.35483869910240173 7.509198621846735e-05
```

With add-one smoothing, the bigram model gives a `u` after a `q` probability 0.35 rather than 1.00. There are only 32 `q`s to count, so the 61 added ones are a large part of the row. The pairs the training text never shows now get probabilities of at least 1 in 13,317.

3 Scoring a language model

3.1 Average log probability

Exercise 3.1 (Scoring held-out text)**.** For each validation character after the first, take the natural log of the probability the smoothed bigram model of Example 2.3 gives it, given the character before it. What is the average of the negatives of these logs?

Solution

Solution 3.1.

```
bigram_log_probs = torch.log(bigram[val_ids[:-1], val_ids[1:]])
bigram_nll = -bigram_log_probs.mean().item()
uniform_nll = math.log(n_chars)
print(bigram_nll, uniform_nll)
```

```
2.3827016353607178 4.110873864173311
```

```
# The prose below says the bigram model's average is the lower one.
assert bigram_nll < uniform_nll
```

The average is 2.38. The uniform model of Example 2.1 gives every character probability $1/61$, so its average is $\log 61 = 4.11$. The lower average means the bigram model gives the validation text higher probability. This average is the cross-entropy loss[a] of the model's predictions of the validation characters.

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-cross-entropy-loss

Definition 3.1 (Perplexity)**.** The **perplexity** of a language model on a sequence $x_1, \dots, x_n$ is

$$\exp\left\{-\frac{1}{n}\sum_{t=1}^{n}(\log\{\Pr(X_t = x_t \mid x_1, \dots, x_{t-1})\})\right\},$$

the exponential of the average negative log probability per symbol [3, Sec. 3.3]. It equals the probability of the whole sequence raised to the power $-1/n$, and it is the same number when the logs are base 2 and the exponential is $2^{(\cdot)}$. The DATA 471 notes define it for any classifier in the same way[a], and find that a model which gives probability $1/K$ to each of K symbols has perplexity K. Lower is better.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#eq-perplexity

Example 3.1 (Perplexity of the two counting models)**.**

```
bigram_ppl = math.exp(bigram_nll)
uniform_ppl = math.exp(uniform_nll)
print(bigram_ppl, uniform_ppl)
```

```
10.834133238705657 61.00000000000001
```

On the validation text after its first character, the uniform model of Example 2.1 has perplexity 61.0, the number of distinct characters. The smoothed bigram model has perplexity 10.8: on average the bigram model is as unsure of the next character as a model choosing evenly among about 11 characters.

Read

Deep Learning's section 12.4[a] covers n-gram and neural language models [2].

[a] https://www.deeplearningbook.org/contents/applications.html

4 A recurrent language model

4.1 The network

```
seq_len = 64
batch_size = 32
lm_lr = 0.005


class CharModel(nn.Module):
    # Embeds each character, reads the sequence with a GRU,
    # and returns logits over the next character at every position.

    def __init__(self, embed_size=16, hidden_size=128):
        super().__init__()
        self.embed = nn.Embedding(n_chars, embed_size)
        self.recurrent = nn.GRU(embed_size, hidden_size, batch_first=True)
```

```
        self.out = nn.Linear(hidden_size, n_chars)

    def forward(self, x, state=None):
        states, state = self.recurrent(self.embed(x), state)
        return self.out(states), state


def make_lm(seed=1):
    torch.manual_seed(seed)
    return CharModel()


def train_lm(net, n_steps, seed=0):
    # Adam on random windows of the training text, with gradient clipping to 1.
    # Each window's targets are the same window moved one character on.
    optimizer = torch.optim.Adam(net.parameters(), lr=lm_lr)
    generator = torch.Generator().manual_seed(seed)
    net.train()
    for step in range(n_steps):
        starts = torch.randint(0, n_train - seq_len - 1, (batch_size,), generator=generator)
        x = torch.stack([train_ids[s:s + seq_len] for s in starts])
        y = torch.stack([train_ids[s + 1:s + seq_len + 1] for s in starts])
        logits, _ = net(x)
        loss = nn.functional.cross_entropy(logits.reshape(-1, n_chars), y.reshape(-1))
        optimizer.zero_grad()
        loss.backward()
        _ = nn.utils.clip_grad_norm_(net.parameters(), 1)
        optimizer.step()


def lm_log_probs(net):
    # Natural log of the probability `net` gives each validation character
    # after the first, given all the validation characters before it.
    net.eval()
    with torch.no_grad():
        logits, _ = net(val_ids[:-1].unsqueeze(0))
    log_p = torch.log_softmax(logits[0], dim=1)
    return log_p[torch.arange(len(val_ids) - 1), val_ids[1:]]


print(make_lm())
```

```
CharModel(
  (embed): Embedding(61, 16)
  (recurrent): GRU(16, 128, batch_first=True)
  (out): Linear(in_features=128, out_features=61, bias=True)
)
```

`CharModel` reads a sequence of character indices with an embedding layer and a GRU layer, and maps the hidden state[3] at each position to logits over the next character. `train_lm()` runs Adam[4] with learning rate[5] 0.005 on batches of 32 windows of 64 characters, with gradient clipping[6] to 1. `lm_log_probs()` returns the log of the probability the network gives each validation character, given the validation characters before it.

[3] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-hidden-state
[4] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-adam
[5] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#def-learning-rate
[6] https://morrison-lab.github.io/lds/chapters/backpropagation.html#def-gradient-clipping

4.2 Predicting every next character

Exercise 4.1 (A recurrent language model)**.** Train `make_lm()` for 400 steps with `train_lm()`. Is its perplexity on the validation text lower than the bigram model's?

Solution

Solution 4.1.

```
lm = make_lm()
untrained_ppl = math.exp(-lm_log_probs(lm).mean().item())
train_lm(lm, 400)
gru_ppl = math.exp(-lm_log_probs(lm).mean().item())
print(untrained_ppl, gru_ppl)
```

```
62.06655063632067 5.192073376728205
```

```
# The prose below says the GRU beats the bigram model; stop the render if not.
assert gru_ppl < bigram_ppl
```

Yes. After training, the network's perplexity on the validation text is 5.2, against 10.8 for the bigram model. Before training it was 62.1, close to the 61 of the uniform model. The bigram model sees one earlier character; the network's hidden state[a] can carry information from every earlier character.

[a] https://morrison-lab.github.io/lds/chapters/cnn-rnn.html#def-hidden-state

4.3 Inputs and targets

Exercise 4.2 (What the network is asked to predict)**.** `train_lm()` asks the network, at each position of a window, for the character that comes next in the text. For a window holding the first 20 characters of the text, what are the 20 targets?

Solution

Solution 4.2.

```
window_x = decode(train_ids[0:20])
window_y = decode(train_ids[1:21])
print(repr(window_x))
```

```
'First Citizen:\nBefor'
```

```
print(repr(window_y))
```

```
'irst Citizen:\nBefore'
```

The targets are characters 2 to 21 of the text: the window moved one character on. The second printed line is the target for each character of the first.

Definition 4.1 (Teacher forcing)**. Teacher forcing** trains a recurrent language model on a text by feeding it the true text as input, and asking it at each position for the next symbol of the same text [2, Sec. 10.2.1]. The targets are the inputs moved one position on. The model never reads its own predictions during training.

Example 4.1 (The loss on one window)**.**

```
x = train_ids[0:seq_len].unsqueeze(0)
y = train_ids[1:seq_len + 1].unsqueeze(0)
logits, _ = lm(x)
window_loss = nn.functional.cross_entropy(logits.reshape(-1, n_chars), y.reshape(-1)).item()
print(logits.shape, y.shape, window_loss)
```

```
torch.Size([1, 64, 61]) torch.Size([1, 64]) 1.252803087234497
```

`train_lm()` uses teacher forcing. For one window of 64 characters, the network returns 64 rows of 61 logits, one row per position, and the cross-entropy loss[a] compares each row with the true next character. For the first window of the text, the trained network's average loss is 1.25.

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-cross-entropy-loss

5 Generating text

5.1 Sampling

Exercise 5.1 (Writing new lines)**.** Draw 1,000 characters from the trained network of Solution 4.1 with `sample(lm, "MENENIUS:\n", 1000)`. What share of the drawn words occur somewhere in the training text?

```
def sample(net, prompt, n, temperature=1.0, seed=0):
    # Feeds `prompt`, then draws `n` characters one at a time,
    # feeding each one back in.
    generator = torch.Generator().manual_seed(seed)
    x = torch.tensor([[index[ch] for ch in prompt]])
    drawn = []
    net.eval()
    with torch.no_grad():
        logits, state = net(x)
        for _ in range(n):
            p = torch.softmax(logits[0, -1] / temperature, dim=0)
            nxt = torch.multinomial(p, 1, generator=generator)
            drawn.append(nxt.item())
            logits, state = net(nxt.view(1, 1), state)
    return prompt + decode(torch.tensor(drawn))


print(sample(make_lm(), "MENENIUS:\n", 40))
```

```
MENENIUS:
xClm.&?;AfTRSGsIvewantsvqxEjtHWvjpyiHCnd
```

`sample()` draws each character from the network's distribution over the next character, given the prompt and the characters drawn so far; the untrained network above draws characters close to at random.

Solution

Solution 5.1.

```
train_words = set(re.findall(r"[a-z]+", text[:n_train].lower()))
prompt = "MENENIUS:\n"


def real_word_share(s):
    # Share of the words drawn after the prompt that occur in the training text.
    words = re.findall(r"[a-z]+", s[len(prompt):].lower())
    return sum(w in train_words for w in words) / len(words)


drawn = sample(lm, prompt, 1000)
drawn_share = real_word_share(drawn)
# The prose below says the draw has speaker names on lines of their own.
assert re.search(r"^[A-Z]+:$", drawn[len(prompt):], re.MULTILINE)
print(drawn[:300])
```

```
MENENIUS:
Well. be prest I am neme
MENy seoplin a diseeals:
He'd that in thriery ssy every more.

VALERIA:
In with ank untemt the payy, what wisds,
The ay opatues my proves to the aween his

MENIUS:
Why with one ourther it Rome he'll
Than stable
Let matters yourselave for tiee ageise their noblie be
```

```
print(drawn_share)
```

```
0.5645161290322581
```

Counting a word as a run of letters, ignoring case, 56% of the drawn words occur in the training text. The rest do not occur there; many of them are not English words. Like the play, the draw puts names in capitals followed by a colon on lines of their own.

i Read

Karpathy [4] trains character-level recurrent language models on Shakespeare, Wikipedia and source code, and shows what they write.

5.2 Temperature

Exercise 5.2 (Dividing the logits)**.** Take the logits $\underset{\sim}{z} = (2, 1, 0)$. Of the softmax[a]es of $\underset{\sim}{z}/0.5$, $\underset{\sim}{z}$ and $\underset{\sim}{z}/2$, which gives the first symbol the most probability?

[a] https://morrison-lab.github.io/lds/chapters/multiclass-logistic-regression.html#def-softmax

Solution

Solution 5.2.

```
z = torch.tensor([2.0, 1.0, 0.0])
divisors = [0.5, 1.0, 2.0]
softened = [torch.softmax(z / d, dim=0) for d in divisors]
first = [p[0].item() for p in softened]
print(softened)
```

```
[tensor([0.8668, 0.1173, 0.0159]), tensor([0.6652, 0.2447, 0.0900]), tensor([0.5065, 0.3072, 0.1863])]
```

```
# The prose below says dividing by 0.5 gives the first symbol the most.
assert first[0] > first[1] > first[2]
```

Dividing by 0.5 gives the first symbol probability 0.87, against 0.67 for z itself and 0.51 for $z/2$. Dividing by a number below 1 spreads the logits apart, which moves probability to the largest; dividing by a number above 1 brings them together, which moves the probabilities toward 1/3 each.

Definition 5.1 (Temperature)**.** Sampling at **temperature** $T > 0$ draws the next symbol from the softmax[a] of z/T, where z is the vector of the model's logits. Hinton et al. [5, Sec. 2] use this softmax with temperature T, and note that a higher T gives a softer distribution. At $T = 1$ sampling uses the model's own distribution. As T falls toward 0, the distribution puts more of its weight on the most likely symbols, and as T grows, the distribution approaches the uniform distribution.

[a] https://morrison-lab.github.io/lds/chapters/multiclass-logistic-regression.html#def-softmax

Example 5.1 (Drawing at three temperatures)**.**

```
temperatures = [0.5, 1.0, 1.5]
draws = [sample(lm, prompt, 1000, temperature=t) for t in temperatures]
shares = [real_word_share(d) for d in draws]
print(shares)
```

```
[0.8781725888324873, 0.5645161290322581, 0.34146341463414637]
```

```
# The prose below says the share falls as the temperature rises,
# from most of the words at the lowest to under half at the highest.
assert shares[0] > shares[1] > shares[2]
assert shares[0] > 0.5 > shares[2]
print(draws[0][:200])
```

```
MENENIUS:
What he show the eward the consers the discess,
We all the city the wars your actrems.

VOLUMNIA:
I will you are my tribunes, and and to the say
That him such as live the well his shall more
```

```
print(draws[2][:200])
```

```
MENENIUS:
Well. but you sevewayts?

LARTWerpyis no.
Yeeals you ar kaquince?
N uf ssy every more.
Of power in reveraus asd vat thibp syreants fores,
Throw.

FilutherI: &oos,
Nor shear enVybelight sood.
```

Temperature	Share of drawn words found in the training text
0.5	88%
1.0	56%
1.5	34%

At temperature 0.5 most of the network's words are ones it has seen. At 1.5 most of its words are not in the training text, and the spelling breaks down.

6 References

[1] A. Karpathy, "Char-rnn: Multi-layer recurrent neural networks for character-level language models." GitHub repository, 2015. Available: https://github.com/karpathy/char-rnn

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[3] D. Jurafsky and J. H. Martin, "Speech and language processing." Draft of the 3rd edition, August 19, 2026, 2026. Available: https://web.stanford.edu/~jurafsky/slp3/

[4] A. Karpathy, "The unreasonable effectiveness of recurrent neural networks." Blog post, 2015. Available: https://karpathy.github.io/2015/05/21/rnn-effectiveness/

[5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network." 2015. Available: https://arxiv.org/abs/1503.02531